#2 If this node is `null`, returns `null` to indicate the key wasn't found.
#3 If the node's key matches the target key, we have found our target: just return current node.
#4 Check how target key compares to current node's.
#5 If it's smaller than current node's key, we need to traverse the left branch.
#6 Otherwise, the target key can only be stored in the right branch, and we need to traverse it.

The API method `contains` for the `Treap` class just calls method search on the root, and returns `false` or `true` depending on whether the result is `null` or not.

3.3.4 Insert

While searching a key in a treap is relatively easy, inserting a new entry, instead, is a completely different story; as we mentioned in section 3.3.1, using BST insertion won't work in the general case, because while the new entry's key would end up in the right place in the tree, its priority might violate heap's invariants, being larger than its parent.

There is no reason to despair, though! We have a way to fix heap's invariants, and we actually have already seen what's the solution: performing a rotation on the node violating the priority constraints.

At a high level, the insert method has just two steps: insert the new node as a leaf, and then check if its priority is higher than its parent; if that's the case, we need to bubble the new node up, but we can't just swap it with its parent, like we would do in a heap.